KOGNIVERA HACKATHON 2026 · TEAM TECH4CE

An AI travel agent that plans boldly and spends carefully.

Natural language in. A transparent plan, a hard spending cap, and a human’s explicit “yes” before a single rupee moves.

THE PROBLEM

Planning a trip is death by a thousand small decisions.

Fifteen browser tabs

Flights here, hotels there, a spreadsheet for the budget nobody keeps updated.

A budget that’s really a guess

Taxes and fees push the real total past whatever number anyone actually agreed to.

Now add an AI agent

Hand the planning to an agent and one question shows up immediately: who’s watching the money?

OUR APPROACH

The agent proposes.
The backend decides.

1. Agent plans

Reads your goal, searches real flights and hotels, drafts a cart.

2. Backend validates

Re-prices everything, adds real tax, checks the cap. Every time.

3. You approve

Only then does a mock booking happen. Never before.

THE PIPELINE

One goal. A checkpoint at every step.

01 Goal received	02 Clarify if needed	03 Plan created	04 Search flights & hotels	05 Budget check	06 Replan if over cap
07 Cart assembled	08 Confirmation requested	09 Human approves	10 Backend re-validates	11 Mock booking	12 Audit recorded

Every one of these twelve steps writes a timestamped, replayable audit event — nothing happens off the record.

ARCHITECTURE

Two processes. One source of truth.

Agent process

`LangGraph · ai/service.py`

Reads a goal, calls tools, drafts a plan. No database, no secrets.



Backend

`FastAPI · backend/src/main.py`

The only authority: pricing, tax, the spend cap, confirmations, bookings.



SQLite

`a restorable working copy`

The original dataset is never touched — verified byte-for-byte, every run.

The agent talks to the backend only over HTTP, with a session-scoped token. It never opens the database — not even to read it.

BUILT WITH

The tools behind it

Python 3.13
The language holding it all together

FastAPI
Human and agent routes, one framework

Pydantic
Validates every request before it touches our logic

SQLite
A restorable working copy, never the original

LangGraph
Orchestrates plan → search → replan

Groq LLM
Optional fast reasoning for messy requests

httpx
How the agent talks to the backend, and only that

Uvicorn
Serves both processes, side by side

pytest / unittest
168 tests, run before every claim we made

HMAC tokens
Session-scoped auth, no external identity provider

Web Push (VAPID)
Trip updates land on your phone, not just the screen

CORS-aware API
A safe boundary for a real frontend to build on

RULE ONE

The spend cap never bends.

Enforced in backend code, not in a prompt. The LLM never sees the final number until the backend has already decided it.

```
effective_cap = MIN(session cap, live cap, trip budget)
committed + (subtotal + 12% tax) ≤ effective_cap
```

Real refusal, from our own demo

Cap ₹25,000 · real total ₹46,638. Refused, replanned three times, then stopped safely — never fabricated an answer.

Same trip, raised cap

Cap ₹60,000 · real total ₹54,503.89 with tax. Passed, cart created, booking confirmed.

RULE TWO

No booking without an explicit yes.

A hash-locked snapshot

The cart is frozen the moment it’s proposed. Approve the wrong hash, and it’s refused.

Human tokens only

We tested it: an agent token trying to approve gets a flat 401. Every time.

Safe to click twice

A repeated approve is refused, not double-booked. We tested that too.

And the instant before a mock booking happens, the backend re-checks the price and the cap one last time — nothing is assumed.

RULE THREE

Every decision leaves a trail you can replay.

`SESSION_CREATED`	A session opens with its cap already stamped in.
`PLAN_CREATED → TOOL_CALL`	The plan is shown before a single search runs.
`BUDGET_CHECK → ACTION_REFUSED`	The exact reason and the exact numbers, recorded.
`REPLAN → CART_CREATED`	A new plan version, a new cart, both on record.
`CONFIRMATION_REQUIRED → HUMAN_CONFIRMED`	The exact moment a person said yes.
`BOOKING_ATTEMPT → MOCK_BOOKING_SUCCESS`	A real reference number, a real timestamp.

Hash-chained end to end. One call, /audit/verify, confirms nothing in the chain has been altered.

PROOF, NOT A PROMISE

We didn’t just build it — we watched it run.

Scenario 1

The exact route requested

No flights exist for it in the dataset. We said so honestly instead of faking a result.

Scenario 2 · over cap

₹25,000 cap, real route

3 real refusals, 3 real replans, then a safe stop. 38 audit events, nothing skipped.

Scenario 3 · within cap

₹60,000 cap, same route

Cart, confirmation, human approval, mock booking — ₹54,503.89, verified end to end.

The original database’s fingerprint was byte-for-byte identical before and after every scenario.

TESTED, NOT ASSUMED

168 tests. Zero shortcuts.

25
Unit tests — money, ids, config

22
Integration tests — the full agent flow

56
Safety tests — DB triggers, isolation

20
End-to-end HTTP tests

45
The agent's own unit tests

168 / 168
passed — 0 failed, 0 skipped

Every database test checks the original file's SHA-256 before and after it runs. It has never once changed.

HOW WE BUILT IT

Two developers, two codebases, one honest merge.

Backend & safety

Built the money math, the confirmation gate, the audit trail, and every rule the agent has to live inside.

Agent & reasoning

Built the LangGraph brain that reads a goal, searches real inventory, and knows when to ask a question.

What actually got in the way

The two pieces evolved apart for days. We reconciled them file by file rather than picking a winner — and along the way, decided the agent's own math would never be trusted as the final word on money.

WHAT'S NEXT

Thank you.

Every agent search over the backend's own catalog, with no exceptions left.

A frontend timeline that renders the audit trail live, as it happens.

Multilingual goal intake, and richer activity recommendations.

Questions?

Team Tech4ce · KogniVera Hackathon 2026